AI Assisted Fusion Workflows

Practical tools for editable timelines and verifiable changes

The most useful next step is to make design history an explicit deliverable: understandable features, meaningful parameters, and evidence that later edits still work.

For new parts, build native parametric history from the start. For an imported solid without history, evaluate AutoTimeline as a reconstruction tool. For either path, use timeline inspection and bounded edits to establish what changed and whether the whole design remains healthy. [1] [10]

The highest value additions

- **AutoTimeline:** a promising way to recover an editable starting point from imported solids; qualify the generated structure before using it as a design baseline.
- **Dedicated timeline inspection:** list features, inspect their model parameters, and report warnings across the complete history.
- **A small edit benchmark:** check a predictable parameter change, including the downstream geometry, before trusting a larger workflow.

Keep three records

Record	Purpose
Fusion timeline	The model's editable sequence of features and relationships.
Validation record	Evidence for one operation: before state, change, checks and outcome.
Saved design version	A named baseline for handoff or recovery.

Research brief | 12 November 2026. Based on official documentation and public repository inspection. Community implementations are evaluation candidates; this brief does not report live Fusion benchmark results.

AutoTimeline for imported solids

AutoTimeline is the most directly relevant new feature to try when a useful model arrives without an editable history.

It predicts a plausible sequence of modeling features from a solid body's boundary representation. Autodesk describes rebuilding candidate sequences and comparing their geometry with the input within tolerance. The result is a possible construction history; the original designer's exact sequence and intent are not uniquely recoverable from shape alone. [3]

Availability and scope

Current Help labels AutoTimeline a Fusion Design Extension Tech Preview. The October 21 release notes say it is available to Design Extension users. A November 3 research article still describes Insider access, so check entitlement and the command in the installed build before scheduling work around it. [1] [2] [3]

The documented input is a supported solid body. Process one body at a time; each generated timeline is saved in a new design. Treat assembly reconstruction and arbitrary mesh input as separate problems. [1]

A practical first trial

1. Select a small imported bracket or plate whose dimensions and intended edits are easy to verify.
2. Use **Design > Solid > Create > Generate Timeline**, or the body's context menu. Follow Job Status, then open the generated design. [1]
3. Compare critical dimensions and geometry against the source. Inspect the feature sequence and identify which dimensions actually drive it.
4. In the separate result, make one intended change and check every downstream feature. Accept it only if the edit behaves as intended.

> **Suggested acceptance criteria:** matching reference geometry within a declared tolerance; intelligible feature names and parameters; no new warnings or errors after recompute; and one successful, representative design change.

Where it belongs in the workflow

Use it for supplier models, legacy parts and geometry received without feature history. Continue creating new designs with explicit parameter expressions and constraints. Merely switching an existing direct design to parametric mode captures its geometry in a Base Feature; that does not reconstruct the earlier operations. [10]

Tools and references worth using

Prioritize tools by the evidence they expose and the edits they preserve. A large command catalog alone says little about whether a model will be easy to revise.

1 Autodesk desktop MCP

The built-in server connects an AI client to a running Fusion session. The documented local endpoint is http://127.0.0.1:27182/mcp, with a configurable port. Use it as the first connection option when it exposes the required operations. A successful connection still needs a separate modeling policy. [4]

2 Autodesk FusionMCPSample

Useful reference architecture for loading context, looking up API details, executing bounded scripts and checking screenshots. It is explicitly a functional sample. Adapt its workflow ideas selectively: the example prompt includes deleting history after a marker, which is unsuitable as automatic recovery for an existing model. [5]

3 hellstein-dev fusion360-mcp

A focused timeline code reference: timeline_list, timeline_health and timeline_feature_parameters expose ordered items, health and editable settings. Its health handler sets ok from the absence of errors; warnings can remain. Borrow the inspection interface, but review compensation and deletion behavior before adopting the writer. [6] [7]

4 faust-machines fusion360-mcp-server

A broader beta implementation with query, parameter, modeling and viewport tools. Its README distinguishes a history-free create_box from create_box_parametric. It also exposes design-type switching and undo. A strict parametric workflow needs a qualified subset, not unrestricted access to every tool. [8]

5 Sketch diagnostics and parameter exchange

The October update's Find Under-Constrained command lists unresolved sketch geometry. User-parameter CSV export provides a useful review companion. Check actual feature expressions too: exporting user parameters alone does not establish that features reference them. [2] [11]

6 Fusion Compute MCP

Autodesk's separate cloud execution option was announced as a public beta in September 2026. Sessions are temporary; outputs need saving or exporting. Consider it for a later, isolated evaluation. Local hooks and local report paths require a new integration contract before this becomes part of a guarded workflow. [9]

> **Best companion reading:** Jeff Strater's Autodesk University guide explains dependency failures, hidden errors while rolled back, and why warnings deserve attention. Start with pages 5-10. [20]

What a useful timeline must preserve

Parameters must reach the features

Create meaningful user parameters with units and intent, then reference them from sketch dimensions and feature settings. Store expressions such as plate_thickness or plate_length - 2 * edge_offset. A parameter table full of names is insufficient if the actual feature distances remain unrelated constants. [11] [18]

Design intent	Expression to preserve
Plate thickness	plate_thickness
Hole spacing along X	plate_length - 2 * edge_offset
Centered mounting points	Symmetry and dimensional constraints
Repeated holes	A native pattern with explicit quantities

Make the construction legible

A useful plate history might be PlateProfile, PlateExtrude, HoleLocation, MountHole and MountPattern. Names explain purpose; groups and component colors help navigation. Organizing the display does not repair broken dependencies. [21]

Handle references and groups explicitly

Treat a timeline index as a position, not a durable identity. Group expansion changes index behavior, and a timeline item may have no public API entity. Traverse group children; a group's Unknown health is not itself a failed feature or proof of healthy contents. [12] [13]

Resolve saved entity tokens using findEntityByToken, then verify the resulting entity's type, owner and geometric role. Token strings can change for the same entity; comparing the strings does not prove object identity. [16]

Check the design after the edit

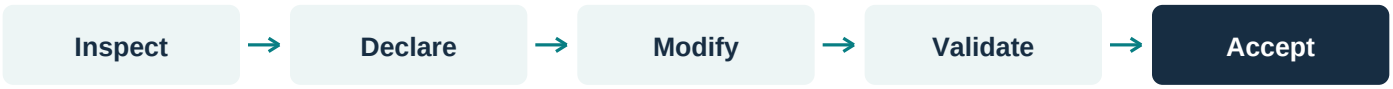
Some edit methods require the marker immediately before their feature; ExtrudeFeature.setOneSideExtent is one documented example. The approved edit must restore the intended final marker position and validate the downstream result. A marker move belongs to the modification step, not to a read-only inspection. [15]

> **Critical API detail:** Design.computeAll() returning true means computation completed. Autodesk explicitly requires checking each timeline item's health to establish whether the features computed successfully. [14]

For a completed-step acceptance check, also verify that the marker is at the end and unexpected features are not suppressed or rolled back. Features beyond the marker can hide downstream failures. [22] [20]

Hooks that validate the timeline

The following is a proposed extension to an existing workflow of native modeling, small steps and acceptance after each change. Keep policy enforcement, model inspection and recovery distinct.



Stage	Required evidence
Before modification	Correct document and component; known baseline; declared operation category; validated target references.
One bounded step	Parameters, one fully constrained sketch, one native feature, or one edit of existing settings.
After modification	Recompute at the intended final state; inspect all timeline health; compare expressions, constraints and geometry.
Acceptance	Complete reports match the operation; expected changes occurred; unexpected changes and new warnings or errors block continuation.
Failure	Preserve the original error and reports; stop writes; inspect without mutation; follow the established recovery process.

Use expected changes as a contract

Declare the expected timeline delta for each operation type. A parameter edit can correctly add zero entries; an edit may change geometry while leaving the count unchanged. Conversely, count growth alone cannot establish that the intended feature was created. Under a strict one-feature rule, operations that create several native entries need a separately reviewed contract.

Keep the enforcement boundary honest

Codex requires trust for non-managed hooks and the project configuration layer. Background hooks cannot block the triggering action; use synchronous hooks for acceptance decisions. A PostToolUse block cannot undo an operation that already ran. [19]

Recommended integration rule: prove the actual call path with a read-only probe and genuine local reports. Keep one writer per document. Changing a server or tool name requires checking hook coverage again. A timeline rollback or deletion of new entries is not a complete recovery of changed parameters, references or other document state.

A small part that tests the workflow

Use a mounting plate to test editability before moving to a complex mechanism. This is a proposed trial, with explicit expected results.

Use a separate test document. Center the plate at the origin: length along X, width along Y, thickness along Z. Place the four hole centers symmetrically about X and Y.

Parameter	Baseline	Intent
plate_length	80 mm	Overall X dimension
plate_width	50 mm	Overall Y dimension
plate_thickness	4 mm	Overall Z dimension
edge_offset	10 mm	Hole-center distance from each adjacent edge
hole_diameter	5 mm	Four through holes

Build and edit

1. Establish the named parameters. Create and fully constrain a centered rectangular sketch.
2. Extrude using plate_thickness. Create and constrain a hole-location sketch, then a native through hole.
3. Create a native rectangular pattern. Use two instances along each axis and expression-driven spacing. Execute each accepted modeling step separately.
4. After accepting the baseline, change only plate_length from 80 mm to 96 mm. Recompute and inspect the entire timeline.

Check	Expected result after the edit
Overall dimensions	96 x 50 x 4 mm
Hole count and diameter	4 holes, each 5 mm
Hole centers	X = -38 or +38 mm; Y = -15 or +15 mm
History	Existing features update; no duplicate replacement features
Constraints and health	Changed sketches remain fully constrained; no new warning or error

Compare native creation with reconstruction

Compare AutoTimeline using a separate imported copy of the baseline. Apply the same geometry and edit checks to its new design. Record how easily the intended dimensions can be changed and what manual cleanup is needed.

Adoption and handoff

Qualify the workflow in three stages

1. Improve the read-only timeline inventory first. Keep unavailable data visible; do not move the marker or recompute in this path.
2. Run the plate edit through the current guarded native API path. Confirm that expressions, measured geometry and acceptance reports agree.
3. Evaluate AutoTimeline and alternative servers in separate documents. Qualify cloud execution separately because its session and evidence storage differ.

Record time to acceptance, manual corrections and failed checks. Qualify perturbation and recovery tests separately in disposable documents before enabling them in routine work.

Record enough to explain a failure

- Document identity, version reference, Fusion build, active component and operation identifier.
- Marker position; ordered item names and types; group membership; suppression and rollback status; per-item health and messages.
- User and model parameter expressions; changed-sketch constraint status and geometry; body count, dimensions and other task-specific invariants.
- Before and after report references, tool outcome and final acceptance state.

For sketches, combine isFullyConstrained with review of dimensions and relationships. A fully fixed sketch can satisfy a status flag while failing the intended design behavior. [17]

Copyable edit prompt

```
Use the established project guard and one writer.
Change only plate_length from 80 mm to 96 mm through
its named parameter expression. Expect existing history
to update without new features. Check a 96 x 50 x 4 mm
plate, four 5 mm holes, and centers at X = +/-38 mm,
Y = +/-15 mm. Require complete post-step acceptance.
On failure or incomplete reports, stop writes, preserve
the original error and follow the recovery process.
```

Handoff package: the native design reference, named parameter table, readable timeline inventory, accepted validation reports, a few annotated views, and a short record of the tested change. The evidence should let the next designer understand both the construction and its demonstrated limits.

Sources and further reading

Primary references reviewed on 12 November 2026. Links in the text open the supporting source. The hellstein-dev implementation links are pinned to the inspected revision; other repository links follow their current default branch.

1. Generate a timeline
Autodesk Help
AutoTimeline workflow and Tech Preview scope.

2. October 2026 product update
Autodesk Fusion Blog
21 October 2026. AutoTimeline and sketch diagnostics.

3. Rebuilding Design History with AutoTimeline
Autodesk Research
3 November 2026. Reconstruction, validation and limits.

4. Connect to the Fusion MCP Server
Autodesk Help
Desktop endpoint and client setup.

5. Unlocking the Power of Fusion MCP
AutodeskFusion360 on GitHub
Reference sample and iterative workflow advice.

6. fusion360-mcp timeline tool definitions
hellstein-dev on GitHub
Inspection, parameter and edit interfaces. Pinned source.

7. fusion360-mcp timeline handlers
hellstein-dev on GitHub
Health semantics and compensation behavior. Pinned source.

8. Fusion360 MCP Server
faust-machines on GitHub
Beta bridge, parametric and direct operations.

9. Fusion Compute MCP Public Beta
Autodesk Fusion Blog
15 September 2026. Separate cloud execution option.

10. Modeling modes in Fusion
Autodesk Help
Parametric history, direct mode and Base Features.

11. Parameters in Fusion
Autodesk Help
User parameters, model parameters and CSV exchange.

12. TimelineObject API reference
Autodesk API
Entities, health, suppression and timeline indices.

13. TimelineGroup API reference
Autodesk API
Child enumeration and group health behavior.

14. Design.computeAll API reference
Autodesk API
Completion is distinct from per-feature success.

15. ExtrudeFeature.setOneSideExtent
Autodesk API
A concrete edit requiring a marker position change.

16. BRepFace.entityToken API reference
Autodesk API
Resolve tokens before comparing entities.

17. Sketch.isFullyConstrained
Autodesk API
Sketch constraint status.

18. ValueInput.createByString
Autodesk API
Unit-aware expressions.

19. Codex hooks
ChatGPT Learn
Trust, synchronous policies and post-tool behavior.

20. Debugging Your Fusion Design
Jeff Strater, Autodesk University
Dependencies and warning diagnosis; especially pages 5-10.

21. Use the Timeline
Autodesk Help
Names, groups, selection sets and component colors.

22. Timeline API reference
Autodesk API
Marker position and timeline collection.